MABJA SOLUTIONS GMBH

Attentio

Serverraumüberwachung (Kamera & Raspberry Pi)

Systemdokumentation

Überwachung eines Serverraumes mittels Personenerkennung durch Kameranutzung und Anbindung eines Raspberry Pis

Architekturdokumentation nach dem arc42-Template (Version 8)
Änderungshistorie nach „Keep a Changelog“ · Entscheidungen als ADRs

Produktname	Attentio
Projektname	Serverraumüberwachung (Kamera & Raspberry Pi)
Projektidee	Überwachung eines Serverraumes mittels Personenerkennung durch Kameranutzung und Anbindung eines Raspberry Pis
Unternehmen	MABJA Solutions GmbH (fiktives Unternehmen im schulischen Projektkontext)
Projektverantwortliche	Anna Porsch Cabrera, Benjamin Sauer, Johana Alarcon Odar, Michael Schertel
Dokument-ID / Version	ATT-SD-001 · Version 1.1 (freigegeben) · 29.09.2026
Systemstatus	Personenerkennung im Betrieb seit 24.09.2026 (Hailo-8L); Datenbank-Anbindung geplant
Klassifizierung	Intern — enthält Netzwerk-Endpunkte, keine Zugangsdaten

Änderungsverzeichnis

Version	Datum	Autor	Änderung
1.0	29.09.2026	MABJA Solutions GmbH	Erstausgabe (englisch): Architektur, Hardware, Deployment, Messergebnisse und Änderungshistorie bis 26.09.2026.
1.1	29.09.2026	MABJA Solutions GmbH	Deutsche Fassung; Einordnung als Projekt „Serverraumüberwachung“ (Produkt Attentio) der MABJA Solutions GmbH; Projektziele mit Zielerreichung ergänzt; Test- und Datenschutzhinweise allgemeiner gefasst.

Inhaltsverzeichnis

1 Einführung und Ziele . . . 4
1.1 Zweck des Dokuments . . . 4
1.2 Projektidee und Aufgabenstellung . . . 4
1.3 Projektziele und Zielerreichung . . . 4
1.4 Funktionale Anforderungen . . . 5
1.5 Qualitätsziele . . . 5
1.6 Stakeholder . . . 6
2 Randbedingungen . . . 6
2.1 Technische Randbedingungen . . . 6
2.2 Organisatorische Randbedingungen . . . 6
2.3 Datenschutz und rechtliche Hinweise . . . 7
3 Kontextabgrenzung . . . 7
3.1 Fachlicher und technischer Kontext . . . 7
3.2 Externe Schnittstellen . . . 7
4 Lösungsstrategie . . . 8
5 Bausteinsicht . . . 8
5.1 Ebene 1 — Container . . . 8
5.2 Ebene 2 — Module von Tracker und OPC-UA-Server . . . 8
6 Laufzeitsicht . . . 9
6.1 Bildverarbeitungs-Pipeline . . . 9
6.2 Lebenszyklus und Leerlaufverhalten . . . 9
6.3 Datenfluss OPC UA . . . 10
6.4 Fehlerszenarien . . . 10
7 Verteilungssicht und Hardware . . . 11
7.1 Verteilung . . . 11
7.2 Hardware . . . 11
7.3 Software-Stack . . . 12
7.4 Ressourcenbudget . . . 12
8 Querschnittliche Konzepte . . . 12
8.1 Latenzoptimierung . . . 12
8.2 Robustheit . . . 13
8.3 Sicherheit . . . 13
8.4 Datenschutz . . . 13
8.5 Beobachtbarkeit . . . 13
8.6 Test- und Prüfvorgehen . . . 13
9 Architekturentscheidungen . . . 14
10 Qualitätsanforderungen und Messergebnisse . . . 15

10.1 Latenz . . . 15
10.2 Durchsatz . . . 15
10.3 Vergleich der Erkennungsvarianten (24.09.2026) . . . 15
11 Risiken und technische Schulden . . . 16
12 Änderungshistorie . . . 17
Phase 1 — Konzept und CPU-Machbarkeitsnachweis (16.09. – 22.09.2026) . . . 17
Phase 2 — Umstieg auf den Hailo-8L (23.09. – 24.09.2026) . . . 17
Phase 3 — Datenschnittstelle (24.09.2026) . . . 17
Phase 4 — Erkennungs- und Tracking-Qualität (24.09.2026) . . . 18
Phase 5 — Betrieb (24.09.2026) . . . 18
Phase 6 — Latenzoptimierung (26.09.2026) . . . 18
Phase 7 — Dokumentation (29.09.2026) . . . 18
Geplante nächste Schritte . . . 18
13 Glossar . . . 19
Anhang A Kurzreferenz für den Betrieb . . . 19

1 Einführung und Ziele

1.1 Zweck des Dokuments

Dieses Dokument beschreibt das System **Attentio** aus dem Projekt **Serverraumüberwachung (Kamera & Raspberry Pi)** der MABJA Solutions GmbH: Zielsetzung, eingesetzte Hardware und Software, Architektur, Laufzeitverhalten, die prägenden Entscheidungen, Messergebnisse und die Entwicklung über die Zeit. Die Gliederung folgt **arc42**, dem verbreiteten Template für Software- und Systemarchitektur-Dokumentation (Kapitel 1–12). Die chronologische Änderungshistorie (Kapitel 12) folgt der Konvention *Keep a Changelog*; wesentliche Entscheidungen sind als *Architecture Decision Records* (Kapitel 9) festgehalten.

Zielgruppe: das Projektteam, Betreiber und Wartende des Systems, alle, die an die Video- oder OPC-UA-Schnittstelle anbinden, sowie Prüfende, die die Architektur nachvollziehen möchten. Die wichtigsten Befehle stehen in Anhang A; die laufend gepflegte Betriebsreferenz ist die Projektdatei `CLAUDE.md`.

1.2 Projektidee und Aufgabenstellung

Projektidee: Überwachung eines Serverraumes mittels Personenerkennung durch Kameranutzung und Anbindung eines Raspberry Pis.

Eine Kamera beobachtet den zu überwachenden Raum und sendet ihr Live-Video an einen Raspberry Pi 5. Dieser erkennt und verfolgt auf einem Hailo-8L-KI-Beschleuniger jede Person im Bild und stellt zwei Ergebnisse bereit:

- ein **annotiertes Live-Video** (Begrenzungsrahmen und Tracking-IDs) zur Ansicht auf berechtigten Geräten und
- **maschinenlesbare Metadaten** — Anzahl erkannter Personen, Kamerastatus und Zeitstempel — über **OPC UA** im Sekundentakt, als Grundlage für Datenbank und Dashboard.

Das System muss aus Netzen funktionieren, die keine eingehenden Verbindungen zulassen und UDP sperren, möglichst wenig Verzögerung verursachen, unbeaufsichtigt laufen, keine Bilder oder Identitäten speichern und sparsam mit den Ressourcen des Raspberry Pi umgehen.

1.3 Projektziele und Zielerreichung

Das Projektteam hat die folgenden Ziele definiert. Stand der Umsetzung: 29.09.2026.

Nr.	Ziel	Status	Umsetzung / Nachweis
Z1	Kamera, die Personen allgemein erkennt	**Erreicht**	Personenerkennung (YOLOv5s auf Hailo-8L) mit Tracking (ByteTrack) in jedem Bild; 30 fps bei 1280×720; annotiertes Live-Video. Siehe Kapitel 6 und 10.
Z2	Jede Erkennung wird in einer Datenbank geloggt	**Teilweise erreicht**	Die Erkennungsdaten werden per OPC UA im Sekundentakt bereitgestellt und wurden mit OPC Router erfolgreich abgerufen. Die Speicherung in einer dedizierten Datenbank folgt mit dem Raspberry Pi mit Kamerasensor (siehe Hinweis).
Z3	Anhand der Datenbank können befugte und unbefugte Zutritte abgeglichen werden	**Geplant**	Setzt Z2 voraus. Vorgesehen ist der Abgleich der protokollierten Erkennungszeitpunkte mit festgelegten Zutrittsberechtigungen (z. B. angemeldete Wartungsfenster) — ohne Identifizierung einzelner Personen.
Z4	In der Datenbank werden Timestamp, Anzahl erkannter Personen und Erkennung (bool) geloggt	**Datenbasis erreicht**	OPC UA liefert `Timestamp` (UTC) und `PersonCount`; die Erkennung (bool) ergibt sich als PersonCount > 0. Zusätzlich `CameraConnected`. Die Speicherung erfolgt mit Z2.

Nr.	Ziel	Status	Umsetzung / Nachweis
Z5	Dashboard zeigt, ob aktuell eine Person erkannt wird und ob die Kamera funktionsfähig ist	**Datenbasis erreicht**	Beide Informationen stehen in Echtzeit per OPC UA bereit (PersonCount, CameraConnected); das annotierte Live-Video ist verfügbar. Die Dashboard-Oberfläche wird zusammen mit der Datenbank-Anbindung umgesetzt.

Hinweis: Die Speicherung der per OPC UA übertragenen Metadaten (Zeitstempel, Anzahl erkannter Personen, Erkennung, Kamerastatus) in einer dedizierten Datenbank wird umgesetzt, sobald der Raspberry Pi mit Kamerasensor in der Schulumgebung bereitgestellt ist. Die OPC-UA-Schnittstelle ist dafür bereits fertig und getestet.

1.4 Funktionale Anforderungen

ID	Anforderung	Status
F-01	Live-H.264-Videostream einer entfernten Kamera auch über Netze annehmen, die eingehende Verbindungen und UDP sperren.	Erfüllt
F-02	Personen in jedem verarbeiteten Bild erkennen und Tracking-IDs vergeben, die stabil bleiben, solange die Person sichtbar ist.	Erfüllt
F-03	Das annotierte Video als RTSP-Stream wiederveröffentlichen.	Erfüllt
F-04	Personenanzahl, Kamerastatus und Zeitstempel per OPC UA mit 1 Hz bereitstellen.	Erfüllt
F-05	Verarbeitung automatisch starten, sobald ein Stream anliegt; Kamera- und Netzausfälle ohne Eingriff überstehen.	Erfüllt
F-06	Metadaten dauerhaft in einer Datenbank speichern und in einem Dashboard anzeigen.	Geplant (Z2, Z5)
F-07	Aufgezeichnete Videodateien offline verarbeiten (Batch-Modus) für Benchmarks und Modellversuche.	Erfüllt

1.5 Qualitätsziele

Prio.	Qualitätsziel	Ziel / Szenario	Erreicht
1	Geringe Latenz	Der annotierte Stream soll sich live anfühlen; der Tracker soll möglichst wenig zur Verzögerung des Kamerastreams beitragen.	≈ 100 ms Kamera-bis-Bildschirm auf /annotated, ≈ 80 ms auf /source (LAN, 26.09.2026)
2	Robuster unbeaufsichtigter Betrieb	Kameraabbrüche, Relay-Aussetzer, Neuverbindungen und Auflösungswechsel ohne manuellen Eingriff überstehen.	5 Tage ohne Absturz oder Speicherwachstum; automatische Wiederverbindung, Leerlauf-Beenden und Neustart
3	Datenschutz durch Technikgestaltung	Keine Aufzeichnung von Video oder Identitäten; nur anonyme Metadaten verlassen das System.	Keine Speicherung von Bildern; OPC UA liefert nur Anzahl, Status und Zeit
4	Ressourcenschonung	Geringer Speicher- und CPU-Bedarf, insbesondere solange kein Videostream anliegt.	≈ 25 MiB und ≈ 0 % CPU im Leerlauf; feste Container-Limits
5	Begrenzte Erreichbarkeit	Nur aus dem privaten Tailnet erreichbar, nie aus dem Internet oder dem lokalen Netz.	Alle Dienste an die Tailscale-Adresse gebunden

1.6 Stakeholder

Rolle	Interesse
Projektteam MABJA Solutions GmbH (Anna Porsch Cabrera, Benjamin Sauer, Johana Alarcon Odar, Michael Schertel)	Funktionsfähige, dokumentierte Lösung für die Serverraumüberwachung; geringer Wartungsaufwand; nachvollziehbare Entscheidungen.
Betreiber des Serverraums	Zuverlässige Erkennung von Anwesenheit im Raum und Nachweis, ob die Kamera funktioniert; Abgleich mit berechtigten Zutritten.
Nutzer von Dashboard und Live-Video	Aktueller Status auf einen Blick; Live-Bild mit geringer Verzögerung in Standard-Playern (VLC, ffplay).
Datenbank / Quell-Pi	Stabiler OPC-UA-Endpunkt mit klar definierten Knoten und eindeutiger Unterscheidung „Kamera offline“ vs. „niemand anwesend“.
Schule (Projektrahmen)	Nachvollziehbare Projektdokumentation; Einhaltung des Datenschutzes.

2 Randbedingungen

2.1 Technische Randbedingungen

Randbedingung	Konsequenz für den Entwurf
Raspberry Pi 5 mit ≈ 8 GB RAM	Feste Container-Limits (Tracker: 3 CPUs / 2 GB, kein Swap); Leerlaufmodus gibt Speicher frei, wenn kein Stream anliegt.
Kein Hardware-Videoencoder im Pi 5	Das annotierte Video wird in Software kodiert (x264 „ultrafast“, „zerolatency“) mit reduzierter Auflösung (746×420).
Eine microSD-Karte mit ≈ 119 GB	Häufig geschriebene Zustände liegen im tmpfs; Container-Logs sind begrenzt.
Der Hailo-8L kann nur von einem Prozess gleichzeitig genutzt werden	Live-Tracker und Batch-Modus laufen nie gleichzeitig; Tests stoppen den Live-Tracker oder verwenden einen Platzhalter-Detektor.
HailoRT-Python-Bindings gibt es nur als apt-Paket für CPython 3.13	Das Tracker-Image basiert auf debian:trixie-slim passend zur Python-ABI des Hosts; hailort 4.23.0 und python3-hailort 4.23.0-1 sind gemeinsam fixiert.
Das Schulnetz sperrt direktes UDP und eingehende Verbindungen	Übertragung per Tailscale mit automatischem Rückfall auf das DERP-Relay über TCP/443; die Bitrate der Kamera ist für Stabilität begrenzt.

2.2 Organisatorische Randbedingungen

- Projekt der fiktiven **MABJA Solutions GmbH** im schulischen Kontext, verantwortet von Anna Porsch Cabrera, Benjamin Sauer, Johana Alarcon Odar, Michael Schertel.
- Der Server (Raspberry Pi 5) wird vom Projektteam bereitgestellt; das Projektverzeichnis ist nicht versioniert. Die Datei `CLAUDE.md` ist die maßgebliche Betriebsdokumentation und wurde bei jeder Änderung nachgeführt.
- Entwicklung und Betrieb erfolgten mit Unterstützung eines KI-Programmierassistenten (Claude Code) direkt auf dem Pi; störende Eingriffe (Neustarts, Produktivwechsel) wurden vorher bestätigt.
- Bis der Raspberry Pi mit Kamerasensor bereitsteht, liefert ein Notebook mit Webcam den Kamerastream.

2.3 Datenschutz und rechtliche Hinweise

Attentio verarbeitet Videobilder ausschließlich im Arbeitsspeicher. Es werden keine Bilder gespeichert und keine Personen identifiziert; das System gibt nur anonyme Metadaten aus (Anzahl erkannter Personen, Kamerastatus, Zeitstempel). Für die Tests wurde ein Videofeed eines Notebooks in einem Klassenraum verwendet. Für den produktiven Einsatz im Serverraum sind die datenschutzrechtlichen Vorgaben (DSGVO) des Betreibers zu berücksichtigen.

3 Kontextabgrenzung

3.1 Fachlicher und technischer Kontext

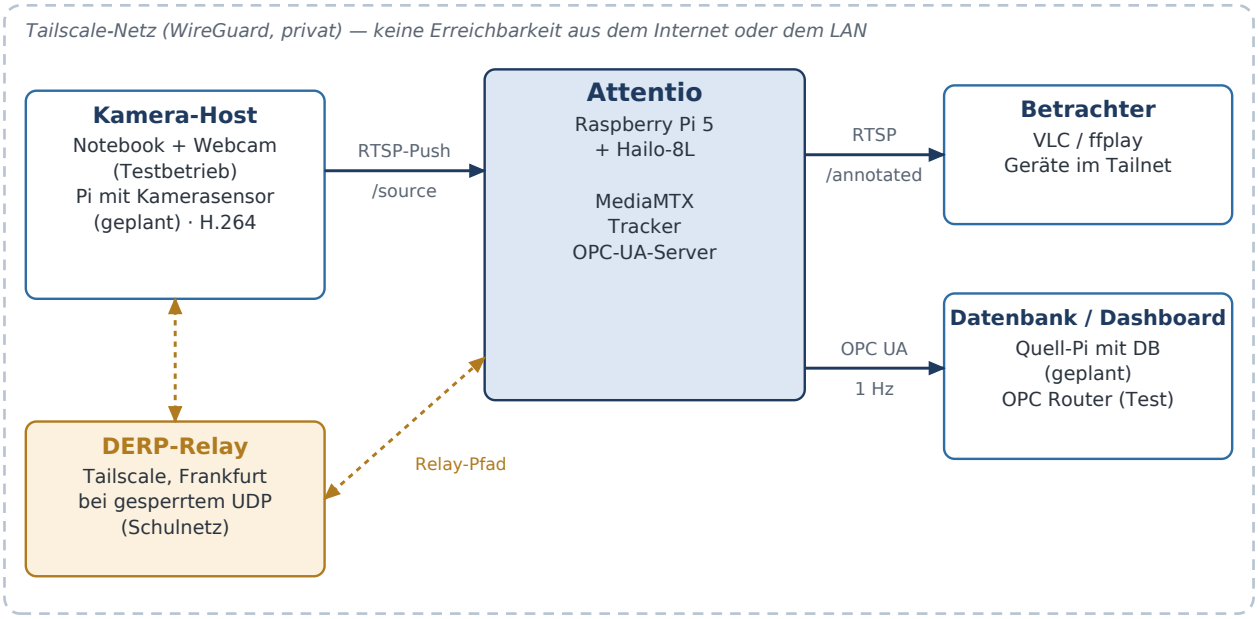


Abbildung 1 — Systemkontext. Alle Verbindungen laufen im privaten Tailscale-Netz; aus dem Schulnetz wird der Kameraverkehr über den DERP-Server von Tailscale in Frankfurt geleitet.

3.2 Externe Schnittstellen

Schnittstelle	Richtung	Protokoll und Endpunkt	Zugriff
Kamera-Eingang	Kamera-Host → System	RTSP über TCP, H.264: `rtsp://100.81.232.86:8554/source`	Publizieren nur mit Zugangsdaten „publisher“
Annotiertes Video	System → Betrachter	RTSP über TCP, H.264 746×420: `rtsp://100.81.232.86:8554/annotated`	Nur lesend, offen im Tailnet
Metadaten	System → Datenbank / Dashboard	OPC UA binär: `opc.tcp://100.81.232.86:4840/camera-tracker/`	Anonym, nur lesend, Sicherheitsmodus None
Transport	alle	Tailscale (WireGuard); im LAN direkt, aus dem Schulnetz über DERP-Relay (TCP/443)	Mitgliedschaft im Tailnet

Nicht im Umfang dieses Systems: Aufnahme und Kodierung auf dem Kamera-Host (dieser muss nur H.264 per RTSP senden), die Datenbank und das Dashboard auf der Quellseite (in Planung, siehe Z2/Z5) sowie die Benutzerverwaltung des Tailnets.

4 Lösungsstrategie

Ziel / Problem	Lösungsansatz
Kamera hinter restriktivem Netz erreichen	Tailscale-VPN: Der Kamera-Host baut nur ausgehende Verbindungen auf; DERP-Relay über TCP/443, wenn UDP gesperrt ist. Keine Portfreigaben, kein öffentlicher Endpunkt.
Erzeuger und Verbraucher von Video entkoppeln	MediaMTX als zentraler RTSP-Knoten mit festen Pfaden (/source, /annotated); jede Komponente ist ein gewöhnlicher RTSP-Client.
Echtzeit-Erkennung auf einem Kleinrechner	Hailo-8L-Beschleuniger mit YOLOv5s-Modell für Personen/Gesichter (57 fps Inferenz) statt CPU-Inferenz (≈ 5 fps).
Stabile Identitäten im Video	ByteTrack-Mehrobjektverfolgung direkt mit den Hailo-Erkennungen, dazu eine Haltezeit von einer Sekunde gegen kurze Aussetzer.
Geringe Latenz	Immer das neueste Bild verarbeiten, Rückstau verwerfen; Dekodierung mit einem Thread; Encoder ohne Verzögerung; Zeitstempel nach Systemuhr.
Industrielle Standardschnittstelle	OPC-UA-Server (asyncua) in eigenem Container, gespeist über einen atomar geschriebenen JSON-Schnappschuss im tmpfs.
Schonender Ressourceneinsatz	Docker Compose mit Ressourcen- und Log-Limits; Leerlaufmodus, der Beschleuniger und Speicher freigibt, wenn kein Stream anliegt.

5 Bausteinsicht

5.1 Ebene 1 — Container

Baustein	Verantwortung	Technologie
mediamtx	RTSP-Knoten. Nimmt den Kamerastream auf /source und den annotierten Stream auf /annotated entgegen und verteilt beide. Authentifizierung für das Publizieren.	bluenviron/mediamtx v1.21.0, nur RTSP/TCP
tracker-live-hailo	Produktiver Tracker. Liest /source, erkennt und verfolgt Personen auf dem Hailo-8L, zeichnet das Overlay, veröffentlicht /annotated und schreibt den Schnappschuss.	Python 3.13, OpenCV 5.0, HailoRT 4.23, ultralytics 8.4 (ByteTrack), ffmpeg 7.1
opcua	OPC-UA-Server. Liest den Schnappschuss einmal pro Sekunde und stellt ihn als drei Variablen bereit.	Python 3.13, asyncua 2.0.1, Host-Netzwerk
tracker (Batch)	Gleiches Image wie der Live-Tracker; verarbeitet eine Videodatei und endet. Start nur bei Bedarf (Compose-Profil „batch“).	wie tracker-live-hailo
Volume tracker-state	Gemeinsames In-Memory-Volume für die Schnappschussdatei `people.json`.	tmpfs, 1 MB

5.2 Ebene 2 — Module von Tracker und OPC-UA-Server

Modul	Inhalt
app/track.py	Einstiegspunkt. Live- und Dateimodus; **LatestFrameReader** (Hintergrund-Thread, der nur das neueste Bild behält und bei Verlust neu verbindet); **RtspWriter** (ffmpeg-Unterprozess zum Kodieren und Publizieren, Neustart bei abgebrochener Pipe); Skalierung und Overlay; Statistikzeile alle 30 s; Beenden nach 120 s ohne Eingang.
app/detectors.py	**HailoDetector**: Letterboxing, optionales Kacheln mit gebündelter Inferenz, NMS-Schwellen zur Laufzeit, Dekodierung der Ausgabe. **_merge_tiles**: fügt Erkennungen an Kachelgrenzen zusammen. **build_tracker/track**: eigenständiges ByteTrack. **TrackHold**: hält eine verlorene Spur bis zu 1 s sichtbar und gezählt.
app/state.py	**PeopleState**: schreibt {written, frame_time, camera_connected, count} zweimal pro Sekunde atomar (temporäre Datei + Umbenennen).

Modul	Inhalt
app/bytetrack_custom.yaml	Tracker-Schwellen: neue Spuren und erster Zuordnungsdurchlauf ab Konfidenz 0,5, zweiter Durchlauf ab 0,2; Spurpuffer 90 Bilder (≈ 3 s bei 30 fps).
app/model.hef	Kompiliertes Hailo-Modell yolov5s_personface_h8l (Klassen Person, Gesicht; NMS im Modell). yolov8s_h8l.hef liegt als getestete Alternative bei.
opcua/server.py	asyncua-Server: Namensraum urn:camera-tracker, Objekt PeopleTracking mit PersonCount, Timestamp, CameraConnected; drei Kompatibilitätsanpassungen für OPC Router (Kapitel 9, ADR-10).
docker-compose.yml, mediamtx.yml, Dockerfile.hailo, Dockerfile.opcua	Deployment-Definition, Konfiguration des RTSP-Knotens und Image-Builds.

6 Laufzeitsicht

6.1 Bildverarbeitungs-Pipeline

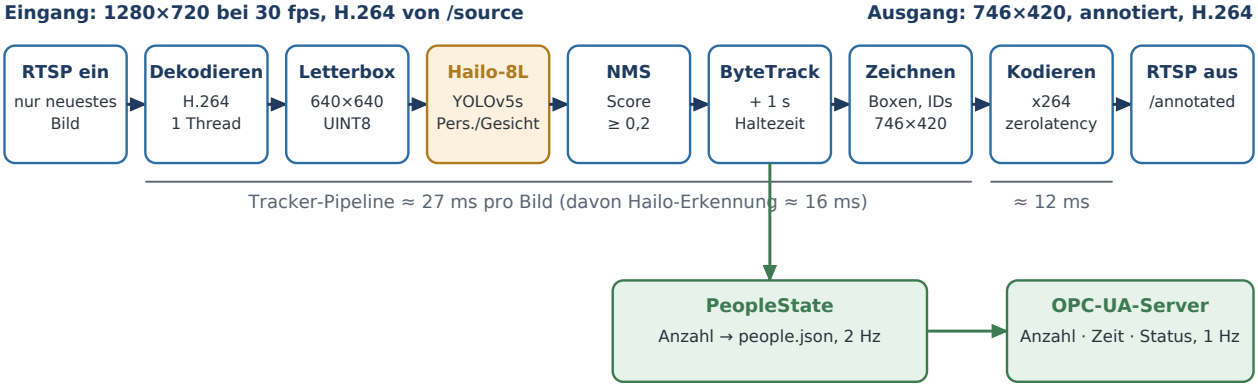


Abbildung 2 — Verarbeitung eines Bildes im Tracker. Die Erkennung läuft auf dem Bild in voller Auflösung; Tracking, Zeichnen und Kodieren nutzen die Ausgabegröße 746×420.

Der Lese-Thread leert die RTSP-Verbindung fortlaufend und behält nur das zuletzt dekodierte Bild; fällt die Verarbeitung zurück, werden Bilder übersprungen statt gepuffert. Jedes verarbeitete Bild wird verzerrungsfrei in den quadratischen 640×640-Eingang des Modells eingepasst (Letterboxing), auf dem Hailo-8L ausgewertet, und die Personen-Erkennungen gehen an ByteTrack. Starke Erkennungen (≥ 0,5) können neue Spuren beginnen; schwächere (0,2–0,5) halten nur bestehende Spuren am Leben, was teilweise verdeckte Personen stabilisiert. Vollständig verpasste Spuren bleiben eine weitere Sekunde an ihrer letzten Position sichtbar. Das annotierte Bild geht ungepuffert an den Encoder, der es mit Systemzeit-Zeitstempeln publiziert.

6.2 Lebenszyklus und Leerlaufverhalten

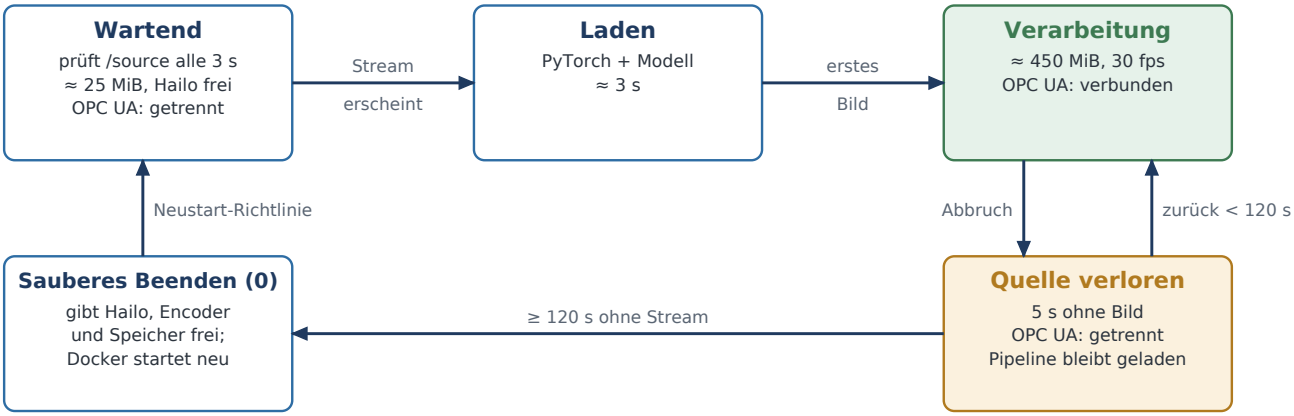


Abbildung 3 — Lebenszyklus des Tracker-Containers. Ein sauberes Beenden mit Neustart pro Kamerasitzung ist beabsichtigt.

Ohne Stream wartet der Tracker als Prozess mit ≈ 25 MiB, der weder PyTorch noch das Modell lädt und den Hailo-8L frei lässt. Sobald ein Stream erscheint, ist in etwa 3 s alles geladen. Kurze Ausfälle (unter zwei Minuten) lassen die Pipeline geladen; nach zwei Minuten ohne Eingang beendet sich der Tracker sauber, und die Neustart-Richtlinie von Docker bringt ihn zurück in den Wartezustand. So werden ≈ 450 MiB frei, die PyTorch innerhalb eines laufenden Prozesses nicht zurückgibt.

6.3 Datenfluss OPC UA

Der Tracker schreibt den Schnappschuss zweimal pro Sekunde; der OPC-UA-Server liest ihn einmal pro Sekunde und aktualisiert alle drei Variablen gemeinsam, mit dem Beobachtungszeitpunkt als *SourceTimestamp*. Ist der Schnappschuss älter als 3 s oder meldet der Tracker die Quelle als verloren, veröffentlicht der Server `CameraConnected = false` und die Anzahl 0 — das bedeutet „keine Beobachtung“, nicht „niemand anwesend“. Der Server speichert keine Historie; die Protokollierung erfolgt in der geplanten Datenbank auf der Quellseite (Z2).

6.4 Fehlerszenarien

Szenario	Verhalten des Systems
Kamerastream bricht ab (Netz, Notebook geschlossen)	Nach 5 s: OPC UA meldet „getrennt“; der Leser verbindet alle 3 s neu. Nach 120 s: sauberes Beenden, Neustart in den Wartezustand.
Kamera verbindet sich mit anderer Auflösung neu	Die Ausgabegröße wird aus jedem Bild bestimmt; Encoder und Tracker werden neu initialisiert (behebt ein gestauchtes Bild vom 24.09.2026).
Ausgabe-Pipe zum Encoder bricht ab	Der ffmpeg-Publisher wird mit 2 s Wartezeit neu gestartet; Bilder werden verworfen, statt den Container abstürzen zu lassen.
Aussetzer auf der Relay-Verbindung	Das Lesen nur des neuesten Bildes verhindert Rückstau; die Bitraten-Begrenzung der Kamera verhindert Pufferüberläufe bei der Aufnahme.
OPC-UA-Client-Sitzung läuft ab	Der Server antwortet mit BadSessionIdInvalid, sodass der Client die Sitzung neu aufbaut; Mindest-Sitzungsdauer auf 120 s angehoben.
Absturz oder hartes Beenden des Trackers	Der Schnappschuss veraltet; innerhalb von 3 s meldet OPC UA „getrennt“. Docker startet den Container neu. Der Hailo wird stets im finally-Block geschlossen, um native Abstürze beim Beenden zu vermeiden.

7 Verteilungssicht und Hardware

7.1 Verteilung

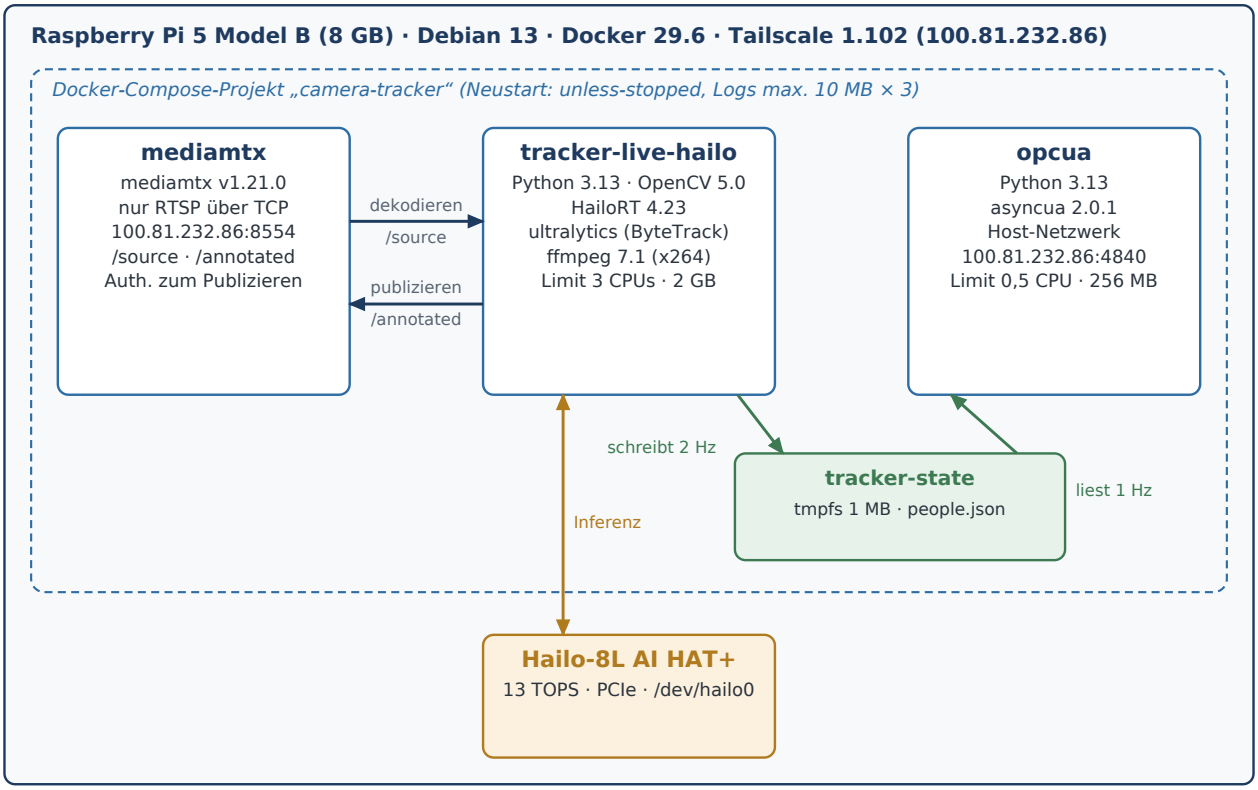


Abbildung 4 — Verteilung auf dem Raspberry Pi 5. MediaMTX und der OPC-UA-Server lauschen nur auf der Tailscale-Adresse.

7.2 Hardware

Komponente	Spezifikation	Rolle
Raspberry Pi 5 Model B Rev 1.1	Broadcom BCM2712, 4 × Arm Cortex-A76 mit 2,4 GHz, ≈ 8 GB LPDDR4X, Gigabit-Ethernet	Server; betreibt das gesamte System
Raspberry Pi AI HAT+ (Hailo-8L)	13 TOPS INT8 KI-Beschleuniger, PCIe über den FFC-Anschluss des Pi 5, Gerät /dev/hailo0; ≈ 41 °C unter Last	Personenerkennung (YOLOv5s, Person/Gesicht)
Speicher	microSD-Karte, ≈ 119 GB	Betriebssystem, Docker-Images (Tracker-Image ≈ 10 GB)
Kamera-Host (Testbetrieb)	Windows-Notebook mit USB-Webcam, ffmpeg (DirectShow, libx264), 1280×720 bei 30 fps, 2,5 Mbit/s	Liefert den Kamerastream an /source
Raspberry Pi mit Kamerasensor (geplant)	Raspberry Pi mit Kamerasensor, Hardware-H.264-Encoder, Tailscale	Künftige Kamera und Träger der Datenbank für die Metadaten
Evaluiert, nicht verwendet	Canon EOS R6 Mark III als USB-Webcam (UVC)	Wurde vom Notebook nicht erkannt; durch Standard-Webcam ersetzt

7.3 Software-Stack

Schicht	Komponente und Version
Betriebssystem	Debian GNU/Linux 13 (trixie), Kernel 6.18 (Raspberry-Pi-Build), aarch64
Container	Docker 29.6.1 mit Docker Compose; Neustart-Richtlinie unless-stopped
Netzwerk	Tailscale 1.102 (WireGuard); Adresse des Pi 100.81.232.86
Video-Knoten	MediaMTX v1.21.0 (nur RTSP über TCP; RTMP, HLS, WebRTC, SRT und MoQ deaktiviert)
KI-Laufzeit	HailoRT 4.23.0, python3-hailort 4.23.0-1, hailo-all 5.1.1, Modell yolov5s_personface_h8l
Tracker	Python 3.13.5, OpenCV 5.0.0, NumPy 2.2.4, ultralytics 8.4.160 (ByteTrack), ffmpeg 7.1.5 mit libx264
OPC UA	Python 3.13, asyncua 2.0.1
Diagnose	hailortcli (Beschleuniger), docker compose ps / logs (Container)

7.4 Ressourcenbudget

Container	Limit	Leerlauf (kein Stream)	Verarbeitung 720p30
tracker-live-hailo	3 CPUs, 2 GB, kein Swap	≈ 25 MiB, ≈ 0 % CPU, Hailo frei	≈ 450 MiB, ≈ 45–65 % eines Kerns
opcua	0,5 CPU, 256 MB	≈ 86 MiB, ≈ 0 % CPU	unverändert
mediamtx	keins	≈ 26 MiB, ≈ 0 % CPU	≈ 30 MiB, ≈ 3–5 % eines Kerns

8 Querschnittliche Konzepte

8.1 Latenzoptimierung

Die Latenz war das wichtigste Qualitätsziel und wurde gemessen statt geschätzt. Die zentralen Techniken:

- **Nur das neueste Bild.** Ein Hintergrund-Leser hält genau ein Bild vor; ein langsamer Verbraucher überspringt Bilder, statt einen Rückstau aufzubauen (ein früherer sequenzieller Leser führte zum Abbruch der Sitzung durch MediaMTX).
- **Dekodierung mit einem Thread.** Die standardmäßige Multithread-Dekodierung von OpenCV hält etwa drei Bilder zurück (≈ 100 ms bei 30 fps). `OPENCV_FFMPEG_THREADS=1` beseitigt das; ein Thread dekodiert 720p noch mit ≈ 280 fps.
- **Kodierung ohne Verzögerung** (x264 ultrafast/zerolatency, Schlüsselbild alle 2 s) mit ungepufferter Pipe zum Encoder.
- **Zeitstempel nach Systemuhr** im Ausgang, damit Betrachter nie hinter die Echtzeit zurückfallen.
- **Einstellungen der Betrachter**: VLC-Netzwerkpuffer 200 ms statt 1000 ms und ein Dekodier-Thread; entsprechende ffplay-Optionen.

8.2 Robustheit

- Automatische Wiederverbindung am Eingang und Neustart des Encoders am Ausgang; kein Fehler im Videopfad beendet den Container.
- Auflösungswechsel werden pro Bild erkannt; Tracker und Encoder werden neu initialisiert.
- Der Beschleuniger wird in einem *finally*-Block freigegeben; ein offen gelassenes Gerät hatte native Busfehler und Core-Dumps verursacht.
- Alle Dienste nutzen die Neustart-Richtlinie *unless-stopped*; der Schnappschuss wird atomar geschrieben.

8.3 Sicherheit

- Alle Dienste sind an die Tailscale-Adresse gebunden; das System ist weder aus dem Internet noch aus dem lokalen Netz erreichbar.
- Publizieren bei MediaMTX erfordert Zugangsdaten (getrennte Benutzer für Kamera und Tracker); Lesen ist innerhalb des Tailnets offen.
- Der OPC-UA-Server ist nur lesend, anonym und ohne Transportsicherheit (Modus None). Das ist nur innerhalb des Tailnets vertretbar; verlassen die Daten das Tailnet, sind Basic256Sha256 mit Zertifikaten und Benutzerauthentifizierung erforderlich.
- Zugangsdaten stehen weder in der Dokumentation noch in Log-Ausgaben (der Tracker maskiert sie).

8.4 Datenschutz

- Videobilder existieren nur im Arbeitsspeicher und werden vom System nicht auf Datenträger geschrieben.
- Das System gibt nur das annotierte Live-Video (ohne Aufzeichnung) und anonyme Metadaten (Anzahl, Status, Zeit) aus.
- Personenbezogene OPC-UA-Daten (Tracking-IDs, Erstsichtungszeiten) wurden am 24.09.2026 entfernt, da sie nicht benötigt werden.
- Für die Tests wurde ein Videofeed eines Notebooks in einem Klassenraum verwendet.

8.5 Beobachtbarkeit

- Der Tracker protokolliert Verbindungswechsel und alle 30 s eine Statistikzeile mit Bildrate und Pipeline-Latenz.
- Logs sind auf 10 MB × 3 Dateien pro Container begrenzt; sich wiederholende Bibliotheksmeldungen (≈ 7.500 Zeilen pro Leerlauftag) sind abgeschaltet.
- Zustand und Logs der Container über `docker compose ps` bzw. `logs`; Zustand des Beschleunigers über `hailortcli`.

8.6 Test- und Prüfvorgehen

Für die Tests wurde ein Videofeed eines Notebooks in einem Klassenraum verwendet. Änderungen wurden vor und nach der Inbetriebnahme geprüft, ohne den laufenden Betrieb zu stören:

- **Isolierte Testumgebungen**: ein separater MediaMTX in eigenem Docker-Netz mit synthetischen Quellen, sodass die produktiven Pfade unberührt bleiben.
- **Latenzmessung per Hell-Dunkel-Wechsel**: Testbilder wechseln sekündlich zwischen Schwarz und Weiß; Messpunkte erfassen, wann der Wechsel auf /source und /annotated erscheint, und trennen so Dekodierung, Verarbeitung und Kodierung.

- **Vergleichstests der Erkennung** auf demselben Ausschnitt des Test-Videofeeds für alle Varianten (Kapitel 10.3).
- **Modultests** für das Zusammenführen von Kacheln und die Haltezeit sowie Nachstellen von OPC-UA-Client-Fehlern auf einem separaten Port vor der Behebung.

9 Architekturentscheidungen

Verdichtete Architecture Decision Records. Status „Angenommen“ bedeutet in Kraft; „Verworfen“ bedeutet geprüft und nicht übernommen.

ADR	Entscheidung	Begründung	Status
01	Tailscale statt Portweiterleitung, DynDNS/Caddy oder WHIP	Das Schulnetz sperrt eingehende Verbindungen und UDP; Tailscale benötigt nur ausgehenden Verkehr und weicht auf TCP/443 aus.	Angenommen
02	MediaMTX als zentraler RTSP-Knoten	Entkoppelt Kamera, Tracker und Betrachter; jede Komponente ist ein einfacher RTSP-Client und verbindet unabhängig neu.	Angenommen
03	Hailo-8L-Beschleuniger; CPU-Variante entfernt	CPU-Inferenz erreichte ≈ 5 fps bei 420p, Hailo-Inferenz 57 fps. CPU-Code, Image und Modell am 24.09.2026 gelöscht.	Angenommen
04	Container-Image auf debian:trixie-slim	HailoRT-Bindings existieren nur als apt-Pakete für das Python 3.13 des Hosts.	Angenommen
05	Eigenständiges ByteTrack, gespeist vom Hailo-Detektor	model.track() von ultralytics setzt eigene Inferenz voraus; der Tracker wird über einen kleinen Adapter direkt angesteuert.	Angenommen
06	OPC UA statt JSON-Dateiexport	Industrieller Standard als Schnittstelle zu Datenbank und Dashboard; ersetzte am selben Tag einen JSON/HTTP-Entwurf.	Angenommen
07	Eigener OPC-UA-Container, Host-Netzwerk, tmpfs-Schnappschuss	Clients bleiben bei Tracker-Neustarts verbunden; korrekt angekündigter Endpunkt; kein Verschleiß der SD-Karte.	Angenommen
08	Letterboxing des Modelleingangs	Das Strecken verzerrte Personen um ≈ 1,8×; Letterboxing verbesserte alle Messwerte.	Angenommen
09	Kacheln, höhere NMS-IoU und yolov8s nur als Optionen	Sie fanden mehr Personen, machten die Anzahl aber unruhiger; Kacheln kosten 2,8× Erkennungszeit.	Verworfen (vorerst)
10	Anpassungen für asyncua 2.0.1 (leerer Browse-Pfad, Sitzungsablauf) und Browse-Namen in ns=2	Nötig für die Kompatibilität mit OPC Router; betrifft private Methoden, bei Updates zu prüfen.	Angenommen
11	Beenden im Leerlauf nach 120 s und verzögertes Laden des Modells	Gibt Beschleuniger und ≈ 450 MiB frei, wenn kein Stream anliegt; ein Neustart pro Sitzung.	Angenommen
12	Dekodierung mit einem Thread	Beseitigt ≈ 100 ms Dekodierverzögerung bei großer Leistungsreserve.	Angenommen
13	Wiedererkennung einzelner Personen	Für die Ziele genügen Erkennung und Anzahl; aus Datenschutzsicht (DSGVO) nicht vorgesehen.	Verworfen

10 Qualitätsanforderungen und Messergebnisse

10.1 Latenz

Messung	Ergebnis	Bedingungen
Kamera bis Bildschirm, /source	≈ 80 ms	LAN über Tailscale-Direktverbindung, VLC mit 200 ms Puffer und 1 Dekodier-Thread (26.09.2026)
Kamera bis Bildschirm, /annotated	≈ 100 ms	gleiche Bedingungen
Vom Tracker verursachte Latenz (Hell-Dunkel-Test)	210 ms → 87 ms	Vor/nach Umstellung auf Dekodierung mit einem Thread; isolierte Umgebung, 1280×720 bei 30 fps
Tracker-Pipeline pro Bild	≈ 27 ms	Dekodiertes Bild bis Encoder, Betriebsstatistik
Hailo-Erkennung	≈ 16 ms	Letterbox-Eingang 640×640, UINT8
x264-Kodierung	≈ 12 ms	746×420, ultrafast/zerolatency
Schulnetz über DERP-Relay (Frankfurt)	≈ 40 ms im Mittel, Spitzen ≈ 250 ms	9,6 Mbit/s dauerhaft, 100 % Zustellung

Latenzverbesserungen nach Wirkung

Änderung	Einsparung	Ort
Netzwerkpuffer des Betrachters 1000 ms → 200 ms	≈ 800 ms	Einstellung Betrachter
Dekodierung mit einem Thread im Tracker	≈ 100–120 ms	track.py
Ein Dekodier-Thread im Betrachter	≈ 100 ms auf beiden Streams	Einstellung Betrachter
Bitraten-Begrenzung der Kamera (2,5 Mbit/s)	Keine Latenzänderung; kein Ruckeln mehr über das Relay	Sende-Befehl
Letterbox und UINT8-Eingang	≈ 2 ms, bessere Erkennung	detectors.py
Ungepufferte Pipe zum Encoder	≈ 3 ms	track.py

10.2 Durchsatz

- Inferenz-Benchmark Hailo-8L: **57,3 fps** bei 640×640 (vorherige CPU-Variante ≈ 5 fps bei 420p).
- Der Betrieb verarbeitet die vollen **30 fps** einer 1280×720-Quelle; die Zeit pro Bild lässt Reserve für mehr als 30 fps.
- Liefert die Webcam selbst weniger Bilder (≈ 12–15 fps bei wenig Licht), passt sich der Tracker der Quellrate an.

10.3 Vergleich der Erkennungsvarianten (24.09.2026)

Alle Varianten liefen auf demselben 90-sekündigen Ausschnitt des Test-Videofeeds (Notebook im Klassenraum) mit gleichen Tracker-Einstellungen und 1 s Haltezeit. „Gefunden“ ist die Zahl angezeigter Spuren pro Bild; unterschiedliche IDs und Änderungen der Anzahl pro Minute messen die Unruhe (je weniger, desto stabiler).

Variante	Erkennungszeit	Gefunden	Unterschiedliche IDs	Anzahl-Änderungen / min
Gestreckter Eingang (vorher)	18,7 ms	7,0	21	46
Letterbox (Betrieb)	16,5 ms	8,5	17	20
Letterbox + NMS-IoU 0,7	16,5 ms	9,2	30	39
Letterbox + 2 Kacheln	45,6 ms	8,9	23	27

Variante	Erkennungszeit	Gefunden	Unterschiedliche IDs	Anzahl-Änderungen / min
Letterbox + 2 Kacheln + IoU 0,7	45,7 ms	10,6	31	48
yolov8s, Letterbox	17,9 ms	8,6	20	22
yolov8s + 2 Kacheln	48,2 ms	8,9	21	29
yolov8s + 2 Kacheln + IoU 0,8	48,3 ms	9,4	23	35

Die Flacker-Unterdrückung (schwache Erkennungen im zweiten ByteTrack-Durchlauf plus 1 s Haltezeit) wurde mit einer simulierten, teilweise verdeckten Person geprüft: Die Lücke in der Ausgabe sank von 9 Bildern auf 0 bei gleichbleibender ID. Zusammen mit dem Letterboxing sanken die Änderungen der Anzahl pro Minute von 46 auf 20.

11 Risiken und technische Schulden

ID	Risiko / Schuld	Auswirkung	Maßnahme
R-01	Anpassungen an asyncua nutzen private Methoden	Ein asyncua-Update könnte die OPC-Router-Korrekturen unbemerkt aufheben.	Version fixiert (2.0.1); Anpassungen bei jedem Update prüfen (dokumentiert).
R-02	ByteTrack-Adapter nutzt ein internes ultralytics-Protokoll	Ein ultralytics-Update könnte das Tracking brechen.	Versionsbereich fixiert; Adapter in detectors.py gekapselt.
R-03	OPC UA ohne Transportsicherheit	Daten für jedes Tailnet-Mitglied lesbar.	Bindung nur an das Tailnet; vor breiterer Freigabe Basic256Sha256 und Benutzer-Authentifizierung.
R-04	Datenbank und Dashboard noch nicht umgesetzt	Ziele Z2, Z3 und Z5 nur teilweise erreicht.	Umsetzung mit Bereitstellung des Raspberry Pi mit Kamerasensor in der Schulumgebung.
R-05	Relay-Verbindung aus dem Schulnetz	Latenzspitzen und schwankender Durchsatz.	Bitraten-Begrenzung; nur neuestes Bild lesen; nach Installation der Kamera neu bewerten.
R-06	Ein einzelner Host mit microSD-Speicher	Ausfall von Hardware oder Karte stoppt den Dienst.	Minimaler Zustand, nur im Speicher; Konfiguration und Dokumentation erlauben den Wiederaufbau.
R-07	Log-Rauschen des Encoders („Non-monotonic DTS“)	≈ 26.000 harmlose Log-Zeilen pro Betriebstag.	Logs begrenzt; Behebung mit Test an einem Live-Betrachter geplant.
R-08	Erkennung an Testmaterial abgestimmt	Die endgültige Kameraposition im Serverraum kann andere Optionen begünstigen.	Vergleichstest mit Material der endgültigen Installation wiederholen.

12 Änderungshistorie

Chronologische Entwicklung des Systems. Kategorien nach *Keep a Changelog*: Hinzugefügt, Geändert, Behoben, Entfernt.

Phase 1 — Konzept und CPU-Machbarkeitsnachweis (16.09. – 22.09.2026)

Datum	Art	Änderung
16.09.	Hinzugefügt	Projektstart auf dem Raspberry Pi 5; Bestandsaufnahme der Umgebung (≈ 8 GB RAM, Docker).
	Geändert	Übertragungsweg festgelegt: Tailscale statt DynDNS/Caddy/WHIP, da das Schulnetz eingehende Verbindungen und UDP sperrt.
	Hinzugefügt	MediaMTX-RTSP-Knoten an der Tailscale-Adresse; authentifiziertes Publizieren; CPU-Tracker mit YOLOv8n und ByteTrack (≈ 5 fps bei 420p).
	Behoben	Zurückfallen der Betrachter: Ausgangszeitstempel auf Systemuhr umgestellt (mit fester Bildrate ergaben 24 s Echtzeit nur 12 s Video).
	Behoben	Sitzungsabbrüche unter Last: LatestFrameReader behält nur das neueste Bild, statt sequenziell zu lesen.
	Hinzugefügt	Log-Begrenzung, nachdem Leerlauf-Logs auf 84 MB angewachsen waren; Schlüsselbild-Intervall für schnellen Start der Betrachter.
22.09.	Geändert	Unbeaufsichtigter Betrieb über 5 Tage ohne Absturz oder Speicherwachstum; CLAUDE.md als Betriebsreferenz etabliert.

Phase 2 — Umstieg auf den Hailo-8L (23.09. – 24.09.2026)

Datum	Art	Änderung
23.09.	Hinzugefügt	Code für einen Beschleuniger vorbereitet (Detektor-Abstraktion, Dockerfile.hailo auf debian:trixie-slim).
23.09.	Hinzugefügt	Hailo AI HAT+ eingebaut; hailo-all installiert; Gerät geprüft; 57,3 fps Inferenz; korrekte Erkennung auf einem Referenzbild.
24.09.	Behoben	Dekodierung der Batch-Dimension; Paketfixierung korrigiert; Hailo wird beim Beenden geschlossen (verhindert Busfehler und Core-Dumps).
24.09.	Geändert	Hailo-Tracker in den Betrieb übernommen.
24.09.	Entfernt	CPU-Variante mit Image, Modell und Code.
24.09.	Behoben	Gestauchtes Bild nach Wechsel der Kamera von 640×480 auf 1280×720: Größe wird nun pro Bild bestimmt.
24.09.	Behoben	Ruckeln und „real-time buffer too full“ über das Schulnetz-Relay: Bitrate der Kamera auf 2,5 Mbit/s begrenzt.

Phase 3 — Datenschnittstelle (24.09.2026)

Datum	Art	Änderung
24.09.	Hinzugefügt	Protokollierung der Erkennungen als JSON-Dateien über HTTP (am selben Tag ersetzt).
24.09.	Geändert	OPC-UA-Server in eigenem Container ersetzt den JSON-Entwurf; Schnappschuss über tmpfs.
24.09.	Behoben	Kompatibilität mit OPC Router: Absturz bei leerem Browse-Pfad, Browse-Namen in Namensraum 2, korrekte Antwort bei abgelaufener Sitzung; Mindest-Sitzungsdauer 120 s.
24.09.	Entfernt	Personenbezogene Knoten (IDs, Erstsichtung); Schnittstelle auf PersonCount, Timestamp, CameraConnected reduziert.

Phase 4 — Erkennungs- und Tracking-Qualität (24.09.2026)

Datum	Art	Änderung
24.09.	Hinzugefügt	Flacker-Unterdrückung: schwache Erkennungen für den zweiten ByteTrack-Durchlauf; 1 s Haltezeit.
24.09.	Geändert	Modelleingang mit Letterboxing und UINT8; Erkennung auf dem Bild in voller Auflösung.
24.09.	Hinzugefügt	Optionen für Kacheln, NMS-IoU und das Modell yolov8s; verglichen und deaktiviert gelassen.

Phase 5 — Betrieb (24.09.2026)

Datum	Art	Änderung
24.09.	Hinzugefügt	Leerlaufmodus: verzögertes Laden des Modells, sauberes Beenden nach 120 s ohne Stream, Neustart in einen 25-MiB-Wartezustand.
24.09.	Geändert	Wiederholte Bibliotheksmeldungen abgeschaltet; Zugangsdaten in Log-Ausgaben maskiert.

Phase 6 — Latenzoptimierung (26.09.2026)

Datum	Art	Änderung
26.09.	Behoben	Vom Tracker verursachte Latenz 210 ms → 87 ms: Dekodierung mit einem Thread (Multithread-Dekodierung hielt ≈ 3 Bilder zurück).
26.09.	Geändert	Ungepufferte Pipe zum Encoder; Empfehlungen für Betrachter (ein Dekodier-Thread, 200 ms Puffer).
26.09.	Geändert	LAN-Test über die Tailscale-Direktverbindung: ≈ 80 ms auf /source, ≈ 100 ms auf /annotated. Latenzarbeiten erfolgreich abgeschlossen.

Phase 7 — Dokumentation (29.09.2026)

Datum	Art	Änderung
29.09.	Hinzugefügt	Systemdokumentation nach arc42 (Version 1.0, englisch) und deutsche Fassung 1.1 mit Projektzielen der MABJA Solutions GmbH.
29.09.	Geändert	Dateiprotokoll von HailoRT abgeschaltet (keine Log-Datei mehr im Projektverzeichnis).

Geplante nächste Schritte

- **Raspberry Pi mit Kamerasensor in der Schulumgebung bereitstellen**: Kameraschnittstelle klären, Hardware-H.264-Encoder mit latenzarmen Einstellungen nutzen (keine B-Frames, kein Lookahead, Schlüsselbild ≈ 1 s, Bitraten-Begrenzung) und gegen die Referenz von 80/100 ms messen.
- **Datenbank (Z2, Z4)**: Metadaten aus OPC UA (Timestamp, Anzahl, Erkennung, Kamerastatus) in einer dedizierten Datenbank speichern.
- **Zutrittsabgleich (Z3)**: Erkennungszeitpunkte mit festgelegten Zutrittsberechtigungen abgleichen, ohne Personen zu identifizieren.
- **Dashboard (Z5)**: aktuelle Erkennung und Funktionsfähigkeit der Kamera anzeigen.
- Vergleichstest der Erkennung mit Material der endgültigen Kameraposition im Serverraum wiederholen.
- OPC-UA-Transportsicherheit ergänzen, falls Daten das Tailnet verlassen.

13 Glossar

Begriff	Erklärung
arc42	Template zur Dokumentation von Software- und Systemarchitekturen in zwölf Kapiteln.
ADR	Architecture Decision Record: kurze Dokumentation einer Entscheidung mit Kontext und Konsequenzen.
ByteTrack	Verfahren zur Mehrobjektverfolgung, das Erkennungen über Bilder hinweg anhand von Bewegung und Überlappung zuordnet und schwache Erkennungen zum Erhalt bestehender Spuren nutzt.
DERP	Relay-Server von Tailscale für Geräte, die keine Direktverbindung aufbauen können; Verkehr über TCP/443.
Hailo-8L / HEF	KI-Beschleuniger (13 TOPS) und dessen kompiliertes Modellformat.
HailoRT	Laufzeitbibliothek und Treiber für Hailo-Beschleuniger.
Letterboxing	Verzerrungsfreies Einpassen eines Bildes in einen quadratischen Eingang mit Auffüllen der Restfläche.
MediaMTX	Medienserver, der RTSP-Streams annimmt und weiterverteilt.
NMS	Non-Maximum Suppression: Entfernen überlappender doppelter Erkennungen.
OPC UA	Open Platform Communications Unified Architecture, Industriestandard für den Datenaustausch zwischen Maschinen (IEC 62541).
RTSP	Real Time Streaming Protocol, hier über TCP für den gesamten Videotransport.
Tailnet / Tailscale	Privates, WireGuard-basiertes Netz, das die Geräte des Projekts verbindet.
tmpfs	Dateisystem, das vollständig im Arbeitsspeicher liegt.
YOLOv5s	Echtzeit-Objekterkennungsmodell; hier eine Variante für die Klassen Person und Gesicht.

Anhang A Kurzreferenz für den Betrieb

Starten, Logs, Neustart (auf dem Raspberry Pi)

```
cd /home/admin/camera-tracker
docker compose up -d                        # mediamtx, tracker-live-hailo, opcua
docker compose logs -f tracker-live-hailo
docker restart camera-tracker-live-hailo    # nach Code-Änderungen (kein Rebuild nötig)
docker compose ps                           # Zustand der Container
```

Kamera einspeisen (Windows-Notebook, ffmpeg)

```
ffmpeg -f dshow -video_size 1280x720 -framerate 30 -i video="KAMERANAME" \
  -c:v libx264 -preset ultrafast -tune zerolatency \
  -b:v 2500k -maxrate 2500k -bufsize 2500k -g 30 \
  -f rtsp -rtsp_transport tcp rtsp://publisher:<passwort>@100.81.232.86:8554/source
```

Mit geringer Verzögerung ansehen

```
vlc --network-caching=200 rtsp://100.81.232.86:8554/annotated
  (dazu Einstellungen > Alle > Eingang/Codecs > Videocodecs > FFmpeg > Threads = 1)
ffplay -fflags nobuffer -flags low_delay -framedrop -threads 1 \
  -rtsp_transport tcp rtsp://100.81.232.86:8554/annotated
```

OPC-UA-Knoten (Namensraum urn:camera-tracker, Index 2)

NodeId	Typ	Bedeutung
ns=2;s=PeopleTracking.PersonCount	Int32	Aktuell verfolgte Personen, inklusive bis zu 1 s verpasster; Erkennung (bool) = PersonCount > 0
ns=2;s=PeopleTracking.Timestamp	DateTime (UTC)	Beobachtungszeitpunkt des Schnappschusses; ändert sich jede Sekunde (als 1-Hz-Auslöser nutzbar)
ns=2;s=PeopleTracking.CameraConnected	Boolean	False, wenn der Stream fehlt oder der Tracker nicht läuft (Anzahl dann 0)

Hinweis: Datenänderungs-Abonnements melden nur Änderungen; für eine gleichmäßige Aktualisierung mit 1 Hz `Timestamp` abonnieren. Alle Zeiten sind UTC und müssen vom Client in Ortszeit umgerechnet werden.